CS 432: Databases

FixIIT – Maintenance Portal

Assignment 3 - Module A

Transaction Management, ACID Validation, and Crash Recovery

Submitted by: Team Codebase

Abhitej Singh Bhullar	(23110009)
Shiv Patel	(23110302)
Shivansh Shukla	(23110303)
Soham Shrivastava	(23110315)
Arin Mehta	(23110038)

GitHub Repository Link:
https://github.com/ArinMehta/FixIIT

Module A Video Link:
Link

Semester II (2025–2026)
Indian Institute of Technology Gandhinagar
Date of Submission: 5 April 2026

Contents

1 Project Objective **2**

2 System Design Summary **2**

2.1 Relations Used 2

2.2 Transaction Engine 2

2.3 WAL and Recovery Model 2

3 Execution Commands **3**

4 ACID Validation Workflow and Results **3**

4.1 Initial Database State 3

4.2 Atomicity Validation 4

4.3 Consistency Validation 6

4.4 Durability Validation 7

4.5 Isolation Validation 9

4.6 WAL Verification 9

5 Failure Handling and Recovery **10**

5.1 Edge Case Coverage 12

6 How Correctness Is Ensured **12**

7 How Failures Are Handled **13**

8 How Multi-User Conflicts Are Handled **13**

9 Experiments Performed **13**

10 Observations and Limitations **14**

11 Conclusion **14**

1 Project Objective

This report presents Module A of Assignment 3, focusing on correctness and reliability of a custom B+ Tree-based database engine under both normal operation and failure scenarios. The implementation validates ACID guarantees for multi-relation transactions and formal crash recovery through write-ahead logging (WAL).

2 System Design Summary

2.1 Relations Used

- members(member_id, name, email, contact_number, age)
- tickets(ticket_id, title, member_id, category_id, priority, status_id)
- assignments(assignment_id, ticket_id, technician_member_id, assigned_by, instructions)

2.2 Transaction Engine

The engine stages operations inside a transaction context, validates candidate state, writes WAL records, applies changes to B+ Trees, and either commits fully or restores a pre-commit snapshot on failure.

2.3 WAL and Recovery Model

The write-ahead log (WAL) stores transaction records that support crash recovery. For each APPLY operation, the log keeps the record state before the change (before-image) and after the change (after-image). These log states are used by recovery to undo incomplete transactions and redo committed transactions after restart.

- **Before-image:** record state immediately before an update/delete operation
- **After-image:** record state immediately after an insert/update operation
- **Recovery role:** undo incomplete transactions and redo committed transactions during restart

The WAL stream contains BEGIN, APPLY, COMMIT, and ABORT records, which together define transaction lifecycle and recovery semantics.

3 Execution Commands

```
cd <REPO_ROOT>/Assignment_3/Module_A
Remove-Item -Path "custom_engine_state" -Recurse -Force -
   ErrorAction SilentlyContinue
python .\run_acid_demo.py
python .\run_edge_cases.py
```

To avoid clashes from earlier runs, we first remove previously persisted records in `custom_engine_state`. After that, we execute `run_acid_demo.py` for the core ACID evidence and `run_edge_cases.py` for additional failure and recovery validation.

4 ACID Validation Workflow and Results

4.1 Initial Database State

We begin by running the combined ACID script on a clean state. The first output block shows the baseline records in the three relations (members, tickets, and assignments) before any transaction experiment is executed.

```
(.venv) C:\Users\shiva\OneDrive - iitgn.ac.in\Desktop\FixIIT-main\Assignment_3\Module_A>python run_acid_demo.py
=== Initial State (FixIIT Schema: members, tickets, assignments) ===
{
  "members": [
    {
      "member_id": 1,
      "name": "Prof. XYZ",
      "email": "prof.xyz@iitgn.ac.in",
      "contact_number": "+91 9123456789",
      "age": 37
    },
    {
      "member_id": 2,
      "name": "Shiv Patel",
      "email": "shiv.patel@iitgn.ac.in",
      "contact_number": "+91 8123406789",
      "age": 20
    },
    {
      "member_id": 3,
      "name": "Prof. ABC",
      "email": "prof.abc@iitgn.ac.in",
      "contact_number": "+91 7123456190",
      "age": 42
    },
    {
      "member_id": 17,
      "name": "Electrician A",
      "email": "electrician.a@fixiit.iitgn.ac.in",
      "contact_number": "+91 9000000001",
      "age": 32
    },
    {
      "member_id": 28,
      "name": "Admin",
      "email": "admin@fixiit.iitgn.ac.in",
      "contact_number": "+91 9999999999",
      "age": 50
    }
```

```
 "tickets": [
   {
     "ticket_id": 1,
     "title": "Projector not working",
     "member_id": 1,
     "category_id": 1,
     "priority": "Medium",
     "status_id": 5
   },
   {
     "ticket_id": 2,
     "title": "Power outlet sparks",
     "member_id": 2,
     "category_id": 1,
     "priority": "Emergency",
     "status_id": 2
   },
   {
     "ticket_id": 3,
     "title": "AC not working in Lab",
     "member_id": 3,
     "category_id": 3,
     "priority": "High",
     "status_id": 2
   }
 ],
 "assignments": [
   {
     "assignment_id": 1,
     "ticket_id": 2,
     "technician_member_id": 17,
     "assigned_by": 28,
     "instructions": "Inspect wiring and replace damaged outlet."
   },
   {
     "assignment_id": 2,
     "ticket_id": 3,
     "technician_member_id": 17,
     "assigned_by": 28,
     "instructions": "Check AC unit cooling system."
   }
```

Figure 1: Initial bootstrap state across members, tickets, and assignments before transaction experiments (continued across two terminal screenshots).

4.2 Atomicity Validation

To validate atomicity, we intentionally inject a failure during a multi-relation transaction. The transaction is interrupted in the commit path after partial progress, and the engine must rollback all staged changes.

```
=== Atomicity Test (Injected failure mid-commit) ===
Attempting to assign ticket 2 to technician 17 with admin 28...
[ATOMICITY] Expected failure captured: Injected failure after 2 operation(s)
[ATOMICITY CHECK] State after failure:
{
  "members": [
    {
      "member_id": 1,
      "name": "Prof. XYZ",
      "email": "prof.xyz@iitgn.ac.in",
      "contact_number": "+91 9123456789",
      "age": 37
    },
    {
      "member_id": 2,
      "name": "Shiv Patel",
      "email": "shiv.patel@iitgn.ac.in",
      "contact_number": "+91 8123406789",
      "age": 20
    },
    {
      "member_id": 3,
      "name": "Prof. ABC",
      "email": "prof.abc@iitgn.ac.in",
      "contact_number": "+91 7123456190",
      "age": 42
    },
    {
      "member_id": 17,
      "name": "Electrician A",
      "email": "electrician.a@fixiit.iitgn.ac.in",
      "contact_number": "+91 9000000001",
      "age": 32
    },
    {
      "member_id": 28,
      "name": "Admin",
      "email": "admin@fixiit.iitgn.ac.in",
      "contact_number": "+91 9999999999",
      "age": 50
    }
```

```
  "tickets": [
    {
      "ticket_id": 1,
      "title": "Projector not working",
      "member_id": 1,
      "category_id": 1,
      "priority": "Medium",
      "status_id": 5
    },
    {
      "ticket_id": 2,
      "title": "Power outlet sparks",
      "member_id": 2,
      "category_id": 1,
      "priority": "Emergency",
      "status_id": 2
    },
    {
      "ticket_id": 3,
      "title": "AC not working in Lab",
      "member_id": 3,
      "category_id": 3,
      "priority": "High",
      "status_id": 2
    }
  ],
  "assignments": [
    {
      "assignment_id": 1,
      "ticket_id": 2,
      "technician_member_id": 17,
      "assigned_by": 28,
      "instructions": "Inspect wiring and replace damaged outlet."
    },
    {
      "assignment_id": 2,
      "ticket_id": 3,
      "technician_member_id": 17,
      "assigned_by": 28,
      "instructions": "Check AC unit cooling system."
    }
```

Figure 2: Atomicity validation output: injected failure during commit and post-failure state showing no partial update persisted (all-or-nothing behavior).

4.3 Consistency Validation

After atomicity testing, we execute a valid multi-relation transaction. The commit is accepted only when referential integrity checks pass for all involved records.

```
=== Consistency Test (Multi-relation validation) ===
Valid commit with ticket 3 assignment...
{
  "members": [
    {
      "member_id": 1,
      "name": "Prof. XYZ",
      "email": "prof.xyz@iitgn.ac.in",
      "contact_number": "+91 9123456789",
      "age": 37
    },
    {
      "member_id": 2,
      "name": "Shiv Patel",
      "email": "shiv.patel@iitgn.ac.in",
      "contact_number": "+91 8123406789",
      "age": 20
    },
    {
      "member_id": 3,
      "name": "Prof. ABC",
      "email": "prof.abc@iitgn.ac.in",
      "contact_number": "+91 7123456190",
      "age": 42
    },
    {
      "member_id": 17,
      "name": "Electrician A",
      "email": "electrician.a@fixiit.iitgn.ac.in",
      "contact_number": "+91 9000000001",
      "age": 32
    },
    {
      "member_id": 28,
      "name": "Admin",
      "email": "admin@fixiit.iitgn.ac.in",
      "contact_number": "+91 9999999999",
      "age": 50
    }
  ],
```

```
  "tickets": [
    {
      "ticket_id": 1,
      "title": "Projector not working",
      "member_id": 1,
      "category_id": 1,
      "priority": "Medium",
      "status_id": 5
    },
    {
      "ticket_id": 2,
      "title": "Power outlet sparks",
      "member_id": 2,
      "category_id": 1,
      "priority": "Emergency",
      "status_id": 2
    },
    {
      "ticket_id": 3,
      "title": "AC not working in Lab",
      "member_id": 3,
      "category_id": 3,
      "priority": "High",
      "status_id": 2
    }
  ],
  "assignments": [
    {
      "assignment_id": 1,
      "ticket_id": 2,
      "technician_member_id": 17,
      "assigned_by": 28,
      "instructions": "Inspect wiring and replace damaged outlet."
    },
    {
      "assignment_id": 2,
      "ticket_id": 3,
      "technician_member_id": 17,
      "assigned_by": 28,
      "instructions": "Check AC unit cooling system."
    },
    {
      "assignment_id": 101,
      "ticket_id": 3,
      "technician_member_id": 17,
      "assigned_by": 28,
      "instructions": "Inspect and repair as per priority"
    }
  ]
}
[OK] Consistency check passed: all foreign keys valid
```


Figure 3: Consistency validation output: successful multi-relation commit execution and final state confirmation with valid foreign-key references.

4.4 Durability Validation

To validate durability, the system is reinitialized after commit. The recovered state is then compared with the pre-restart committed state.

```
=== Durability/Recovery Test (Restart DB) ===
Restarting database to verify committed data persists...
{
  "members": [
    {
      "member_id": 1,
      "name": "Prof. XYZ",
      "email": "prof.xyz@iitgn.ac.in",
      "contact_number": "+91 9123456789",
      "age": 37
    },
    {
      "member_id": 2,
      "name": "Shiv Patel",
      "email": "shiv.patel@iitgn.ac.in",
      "contact_number": "+91 8123406789",
      "age": 20
    },
    {
      "member_id": 3,
      "name": "Prof. ABC",
      "email": "prof.abc@iitgn.ac.in",
      "contact_number": "+91 7123456190",
      "age": 42
    },
    {
      "member_id": 17,
      "name": "Electrician A",
      "email": "electrician.a@fixiit.iitgn.ac.in",
      "contact_number": "+91 9000000001",
      "age": 32
    },
    {
      "member_id": 28,
      "name": "Admin",
      "email": "admin@fixiit.iitgn.ac.in",
      "contact_number": "+91 9999999999",
      "age": 50
    }
  ],
```

```
  "tickets": [
    {
      "ticket_id": 1,
      "title": "Projector not working",
      "member_id": 1,
      "category_id": 1,
      "priority": "Medium",
      "status_id": 5
    },
    {
      "ticket_id": 2,
      "title": "Power outlet sparks",
      "member_id": 2,
      "category_id": 1,
      "priority": "Emergency",
      "status_id": 2
    },
    {
      "ticket_id": 3,
      "title": "AC not working in Lab",
      "member_id": 3,
      "category_id": 3,
      "priority": "High",
      "status_id": 2
    }
  ],
  "assignments": [
    {
      "assignment_id": 1,
      "ticket_id": 2,
      "technician_member_id": 17,
      "assigned_by": 28,
      "instructions": "Inspect wiring and replace damaged outlet."
    },
    {
      "assignment_id": 2,
      "ticket_id": 3,
      "technician_member_id": 17,
      "assigned_by": 28,
      "instructions": "Check AC unit cooling system."
    },
    {
      "assignment_id": 101,
      "ticket_id": 3,
      "technician_member_id": 17,
      "assigned_by": 28,
      "instructions": "Inspect and repair as per priority"
    }
  ]
}
[OK] Durability verified: state matches after restart
```

Figure 4: Restart verification confirming committed state persisted across process restart.

4.5 Isolation Validation

Isolation is validated using concurrent transaction ordering. The output below captures both checks in sequence: a read before commit that does not observe uncommitted state, and a read after commit that observes the committed update.

```
=== Isolation Test (No Dirty Reads) ===
[STEP] T1 staged update (uncommitted): priority=Isolation-T1
[STEP] T2 read before T1 commit: {
  "ticket_id": 1,
  "title": "Projector not working",
  "member_id": 1,
  "category_id": 1,
  "priority": "Medium",
  "status_id": 5
}
[OK] Isolation check #1 passed: no dirty read
[STEP] T3 read after T1 commit: {
  "ticket_id": 1,
  "title": "Projector not working",
  "member_id": 1,
  "category_id": 1,
  "priority": "Isolation-T1",
  "status_id": 5
}
[OK] Isolation check #2 passed: committed data is visible after commit

[INFO] Commit critical section is serialized by an RLock in engine.commit().

[INFO] WAL location: C:\Users\shiva\OneDrive - iitgn.ac.in\Desktop\FixIIT-main\Assignment_3\Module_A\custom_engine_state\wal.jsonl
[OK] All ACID checks completed
```

Figure 5: Isolation validation output: no dirty read before commit, and committed state visible after commit.

4.6 WAL Verification

Finally, we inspect the WAL entries generated during execution. Because the log is longer than a single screen, the report includes a representative excerpt rather than the full file. The visible records are sufficient to show the transaction lifecycle (BEGIN, APPLY, COMMIT, ABORT) and the recovery trail required to redo committed transactions and undo incomplete ones.

Assignment_3 > Module_A > custom_engine_state > {} wal.jsonl


```
{"type":"BEGIN","tx_id":"tx-049bd8fdc383","ts":1775384644.1183877}
{"type":"APPLY","tx_id":"tx-049bd8fdc383","ts":1775384644.1230574,"table":"members","key":
{"type":"APPLY","tx_id":"tx-049bd8fdc383","ts":1775384644.1273081,"table":"members","key":
{"type":"APPLY","tx_id":"tx-049bd8fdc383","ts":1775384644.1319413,"table":"members","key":
{"type":"APPLY","tx_id":"tx-049bd8fdc383","ts":1775384644.1355793,"table":"members","key":
{"type":"APPLY","tx_id":"tx-049bd8fdc383","ts":1775384644.1390827,"table":"members","key":
{"type":"APPLY","tx_id":"tx-049bd8fdc383","ts":1775384644.14355,"table":"tickets","key":1,"
{"type":"APPLY","tx_id":"tx-049bd8fdc383","ts":1775384644.1464384,"table":"tickets","key":
{"type":"APPLY","tx_id":"tx-049bd8fdc383","ts":1775384644.1494591,"table":"tickets","key":
{"type":"APPLY","tx_id":"tx-049bd8fdc383","ts":1775384644.1525617,"table":"assignments","k
{"type":"APPLY","tx_id":"tx-049bd8fdc383","ts":1775384644.155274,"table":"assignments","ke
{"type":"COMMIT","tx_id":"tx-049bd8fdc383","ts":1775384644.1578698,"op_count":10}
{"type":"BEGIN","tx_id":"tx-bbf537a0e134","ts":1775384644.2161305}
{"type":"APPLY","tx_id":"tx-bbf537a0e134","ts":1775384644.2191722,"table":"tickets","key":
{"type":"APPLY","tx_id":"tx-bbf537a0e134","ts":1775384644.2219253,"table":"assignments","k
{"type":"ABORT","tx_id":"tx-bbf537a0e134","ts":1775384644.2469308,"reason":"Injected failu
{"type":"BEGIN","tx_id":"tx-585d47819e39","ts":1775384644.2523963}
{"type":"APPLY","tx_id":"tx-585d47819e39","ts":1775384644.2558026,"table":"tickets","key":
{"type":"APPLY","tx_id":"tx-585d47819e39","ts":1775384644.258961,"table":"assignments","ke
{"type":"COMMIT","tx_id":"tx-585d47819e39","ts":1775384644.2617288,"op_count":2}
{"type":"BEGIN","tx_id":"tx-e903e438b74f","ts":1775384644.3325827}
{"type":"BEGIN","tx_id":"tx-97e13059b081","ts":1775384644.3354845}
{"type":"APPLY","tx_id":"tx-e903e438b74f","ts":1775384644.33923,"table":"tickets","key":1,"
{"type":"COMMIT","tx_id":"tx-e903e438b74f","ts":1775384644.3419945,"op_count":1}
{"type":"ABORT","tx_id":"tx-97e13059b081","ts":1775384644.3677368,"reason":"manual_rollbac
{"type":"BEGIN","tx_id":"tx-dc5e99b9d246","ts":1775384644.3704202}
{"type":"ABORT","tx_id":"tx-dc5e99b9d246","ts":1775384644.37345,"reason":"manual_rollback"
```


Figure 6: Representative WAL excerpt showing BEGIN, APPLY, COMMIT, and ABORT records used for transaction recovery.

5 Failure Handling and Recovery

This section summarizes the additional edge-case experiments used to validate failure handling and recovery behaviour beyond the main ACID demonstration. The output shown below is a representative excerpt from `run_edge_cases.py`, which exercises rollback, constraint enforcement, and restart recovery under controlled failures.

```
(.venv) C:\Users\shiva\OneDrive - iitgn.ac.in\Desktop\FixIIT-main\Assignment_3\Module_A>python run_edge_cases.py
=== Module A: Edge Case Testing ===

======================================================================
EDGE CASE 1: Crash After First Write But Before Commit
======================================================================
[SCENARIO] Transaction writes to members table, then crashes before
           writing to tickets table. Expected: all writes undone.

[INITIAL] Members count: 5
[WRITE 1] Inserted member_id=99 (uncommitted)
[CRASH] Simulated crash after first write
[FINAL]  Members count: 5
[PASS] Edge case 1: Partial write was rolled back (atomicity)

======================================================================
EDGE CASE 2: Multiple Writes in Same Transaction, Then Failure
======================================================================
[SCENARIO] Transaction writes to 2 tables successfully, then fails
           before commit phase. Expected: all writes undone.

[INITIAL] Tickets count: 3
[WRITE 1] Inserted ticket_id=99 (uncommitted)
[WRITE 2] Inserted assignment_id=199 (uncommitted)
[CRASH] Simulated crash after multiple writes
[FINAL]  Tickets count: 3
[PASS] Edge case 2: All writes rolled back (atomicity across tables)

======================================================================
EDGE CASE 3: Duplicate Primary Key Insert (Constraint Violation)
======================================================================
[SCENARIO] Transaction attempts to insert duplicate assignment_id.
           Expected: insert rejected, exception raised.

[EXCEPTION] Duplicate key '1' in table 'assignments'
[PASS] Edge case 3: Duplicate key rejected before commit

======================================================================
EDGE CASE 4: Foreign Key Constraint Violation
======================================================================
[SCENARIO] Transaction creates assignment with invalid member_id FK.
           Expected: rejected at commit time, transaction rolled back.

[WRITE] Inserted assignment_id=200 with technician_member_id=999999 (invalid)
[EXCEPTION] Consistency violation: assignments.technician_member_id 999999 does not reference valid member
[PASS] Edge case 4: Foreign key violation rejected at commit

======================================================================
EDGE CASE 5: Rollback on Empty Transaction (Idempotent)
======================================================================
[SCENARIO] Begin transaction, do nothing, then rollback.
           Expected: safe, idempotent, no errors.

[BEGIN] Empty transaction started
[ROLLBACK] Empty transaction rolled back
[PASS] Edge case 5: Empty rollback is idempotent
```

```
======================================================================
EDGE CASE 6: Committed Transaction Survives Multiple Restarts
======================================================================
[SCENARIO] Commit a transaction, restart DB twice, verify state persists.
           Expected: idempotent recovery, same state both times.

[COMMIT] Assignment_id=300 committed
[STATE 1] Assignments count: 4
[RESTART 1] Assignments count: 4
[RESTART 2] Assignments count: 4
[PASS] Edge case 6: Recovery is idempotent across multiple restarts

======================================================================
SUMMARY: Edge Case Coverage
======================================================================

ACID Properties Proven by Edge Cases:

1. ATOMICITY:
   - Edge case 1: Crash after 1st write -> all undone
   - Edge case 2: Crash after N writes -> all undone
   - Edge case 4: FK violation -> rollback

2. CONSISTENCY:
   - Edge case 3: Duplicate key rejected
   - Edge case 4: FK validator enforced
   - Edge case 5: Empty transaction (no data inconsistency)

3. DURABILITY:
   - Edge case 6: Recovery idempotent across restarts
   - Committed deltas persist, uncommitted discarded

4. ISOLATION:
   - (Tested in run_isolation_demo.py)

Assignment Requirements Met:
[OK] Simulate failures during transaction execution (edge cases 1, 2, 4)
[OK] Ensure partial updates are rolled back (edge cases 1, 2)
[OK] Ensure committed data is preserved (edge case 6)
[OK] After restart: undo incomplete transactions (edge cases 1, 2)
[OK] After restart: retain committed transactions (edge case 6)

[INFO] WAL location: C:\Users\shiva\OneDrive - iitgn.ac.in\Desktop\FixIIT-main\Assignment_3\Module_A\custom_engine_state\wal.jsonl
[OK] All edge cases completed
```

Figure 7: Representative edge-case output from `run_edge_cases.py`, showing the complete failure-handling and recovery summary .

5.1 Edge Case Coverage

The edge-case script is structured to cover the main failure modes required by the assignment. It first verifies that partial writes are rolled back, then checks that invalid transactions are rejected, and finally confirms that committed state remains stable across repeated restarts.

Edge Case	**Property Verified**	**Expected Outcome**
Crash after first write	Atomicity	All writes are rolled back.
Crash after multiple writes	Atomicity	No partial state is retained.
Duplicate primary key insert	Consistency	Transaction is rejected before commit.
Foreign key violation	Consistency	Invalid reference is not accepted.
Rollback on empty transaction	Safety / Idempotence	Rollback completes without error.
Committed state after multiple restarts	Durability	Final committed state remains unchanged.

Table 1: Summary of edge cases used to validate failure handling and recovery.

The screenshot captures the final summary produced by the script. Together with the detailed figure captions and the table above, it demonstrates that the engine handles both transaction failure and restart recovery in a controlled and repeatable manner.

6 How Correctness Is Ensured

Correctness is ensured by staging every transaction in memory first and validating the resulting candidate state before any permanent change is accepted. This prevents invalid records from being written into the B+ Tree-backed relations. In addition, the transaction logic applies the complete set of operations together, so a transaction either finishes successfully or is fully rolled back.

In this implementation, correctness depends on three checks: primary-key uniqueness, referential integrity across the three relations, and the absence of partially applied changes. The validation step is performed on the simulated post-state rather than on the live tables, which allows the engine to reject an unsafe transaction before it affects persistent data.

7 How Failures Are Handled

Failures are handled through explicit abort logic, rollback of staged operations, and snapshot restoration of the pre-transaction state. When a failure occurs during commit, the engine does not leave the database in an intermediate state. Instead, it restores the previous state and records the abort in the WAL so the failure is visible during recovery as well.

This approach is important because it protects the database from partial writes caused by crashes or injected errors. The edge-case tests demonstrate that even when a transaction fails after one or more writes, the earlier state remains intact and the unsuccessful transaction does not appear in the final database state.

8 How Multi-User Conflicts Are Handled

Module A handles concurrent access by serializing the commit phase with a lock, which ensures that only one transaction finalizes changes at a time. This avoids conflicting updates to the same records and prevents dirty reads during transaction execution. The isolation experiment shows that a second transaction cannot observe changes that have not yet been committed.

Although this is a simple concurrency strategy, it is sufficient for the assignment because it preserves correctness under concurrent access. The focus here is not maximum throughput, but safe execution without corruption or visibility of uncommitted data.

9 Experiments Performed

1. Atomicity failure injection during multi-relation transaction.
2. Consistency validation through valid and invalid transaction paths.
3. Durability verification by restart-based state comparison.
4. Isolation verification using concurrent transaction ordering.
5. Additional robustness tests via edge-case execution.

The experiments were arranged in a clear order so that each property could be demonstrated separately and then supported by the edge-case script. First, the main ACID run establishes the expected behaviour of a normal transaction under failure and recovery. Then, the edge-case run extends this by showing rollback after partial writes, rejection of invalid inserts, and repeated restart recovery.

10 Observations and Limitations

Observations:

- The combined ACID script provides a clean sequence of outputs for the report and video.
- WAL-based recovery separates committed and incomplete transactions in a predictable way.
- The edge-case script strengthens the report by showing failure handling beyond the main demonstration.

Limitations:

- Isolation is implemented using serialized commits rather than a more advanced multi-version scheme.
- Recovery is local and single-node, so it does not address distributed failures.
- Large-scale performance evaluation is done in Module B, while Module A focuses on correctness.

11 Conclusion

Module A extends the Assignment 2 B+ Tree-based database with transaction management, failure handling, and recovery support. The experiments in this report show that multi-relation transactions can be committed safely, rolled back on failure, and recovered correctly after restart. The system also preserves isolation through serialized commits and rejects invalid updates through pre-commit validation. Overall, the implementation satisfies the ACID requirements of the assignment and provides a reliable base for further concurrency testing in Module B.